KNU 강원대학교
KANGWON NATIONAL UNIVERSITY

R E S U L T R E P O R T

마이크로프로세서응용및실험 결과보고서_입출력포트



수 강 과 목	: 마이크로프로세서응용및실험-2분반
담 당 교 수	: 김도인 교수님
학 번	: 202210820
작 성 자	: 이동제
공 동 실 험 자	: 강민수(202214221)
제 출 일	: 2024-05-02 목요일

<목 차>

1. 실험목적 ····· 2

2. 실험원리 ····· 2

3. 실험기구 및 재료 ····· 3

4. 실험방법 ····· 3-4

5. 측정값 ····· 5

6. 실험결과 ····· 5

7. 결과에 대한 논의 ····· 6

8. 결론 ····· 6

9. 참고문헌 및 출처 ····· 6

1. 실험목적

ATmega128의 Input/Output 포트(Port)를 사용하는 방법을 익힌다. 또한 LED를 구동하는 방법을 배운다.

2. 실험원리

완성된 ATmega128의 마이크로 컨트롤러와 컴퓨터의 AVR Studio라는 프로그램을 통해, LED가 동작할 수 있는 코드를 작성 후 입력(Input) 포트(스위치)에 넣어서 실제로 코드에 입력한 값대로 작동하는지 출력(Output) 포트(LED)를 통해 확인할 수 있다.

입출력 포트는 총 7개로, PA, PB, PC, PD, PE, PF, PG가 있습니다. (입출력 기능 외에 부가기능도 보유하고 있습니다.)

각각의 포트는 독립적인 입출력이 가능한 8비트(1바이트)의 양방향 입출력 단자로 구성됐습니다. 하지만 PG만 5비트입니다.

이제 입출력 포트를 사용하는 방법을 알아야 하는데, 그러기 위해선 레지스터를 통해 포트의 핀을 제어해야 합니다. 3가지의 레지스터로 핀의 데이터 방향을 나타내는 레지스터 DDRx, 출력 데이터를 저장하는 PORTx, 입력 데이터를 저장하는 PINx 이 3가지의 레지스터를 사용합니다.

16진수를 표시하는 경우 0x(Hex)를 상숫값 앞에 붙여주고, 2진수를 표시하는 경우 0b(Binary)를 붙여줍니다.

DDRx : Port x Data Direction Register
각 핀에 해당된 비트에 1을 입력하면 출력 핀이 됩니다. 0을 입력할 경우 입력 핀이 되며, 풀업 저항이 쓰입니다.

PORTx : Port x Data Register
출력일 경우, 각 핀에 해당된 비트에 1을 입력하면 1을, 0을 입력하면 0을 출력합니다.

PINx : Port x Input Pins Address
각 핀에 해당된 비트를 읽어서 핀에 입력된 데이터를 확인할 수 있습니다.

비트연산자
& : 대응되는 비트가 모두 1이면 1을 반환함. (비트 AND 연산)
| : 대응되는 비트 중에서 하나라도 1이면 1을 반환함. (비트 OR 연산)
^ : 대응되는 비트가 서로 다르면 1을 반환함. (비트 XOR 연산)
~ : 비트를 1이면 0으로, 0이면 1로 반전시킴. (비트 NOT 연산)
<< : 지정한 수만큼 비트들을 전부 왼쪽으로 이동시킴. (left shift 연산)
>> : 부호를 유지하면서 지정한 수만큼 비트를 전부 오른쪽으로 이동시킴. (right shift 연산)

이러한 실험 원리들을 가지고 I/O 포트의 핀들을 제어하는 코드를 작성해 LED를 On/Off하는 테스트를 진행합니다.

3. 실험기구 및 재료

ATmega128 마이크로 컨트롤러	1개
컴퓨터	1개
ISP 케이블	1개

표 2) 실험기구 및 재료

4. 실험방법

LED를 테스트하기 위해 AVR Studio 속에서 코드를 작성합니다.

첫 번째로 홀수번째 LED와 짝수번째 LED를 번갈아 가며 1초씩 반복해서 점등하는 코드를 작성합니다. (Code_ODD or Even)

```
#define F_CPU 16000000L
#include <avr/io.h>
#include <util/delay.h>
```

일단 처음은 CPU의 클럭 주파수를 16MHz로 설정시키고, 포트 및 레지스터에 대한 정의를 시키고, 지연(Delay)함수를 사용하기 위해서 AVR 라이브러리 속 헤더 파일을 포함하는 코드를 입력합니다.

```
int main(void)
```

프로그램이 시작할 때 이 함수가 먼저 호출되도록 int를 통해 진입점을 정해줍니다.

```
{
        DDRB = 0xFF;
        while (1)
```

DDRB = 0xFF(0b11111111)를 통해 포트B의 모든 핀을 출력(1)으로 설정합니다. 그리고 while 문을 통해 무한 루프. 즉 무한 반복을 하도록 합니다.

```
        {
                PORTB = 0x55;
                _delay_ms(1000);
                PORTB = 0xAA;
                _delay_ms(1000);
        }
```

0x55는 16진수로 표현된 방식이며 해석하기 쉽도록 2진수로 표현하자면 01010101을 나타냅니다. 이는 짝수 번째 LED만 켜지도록 만들고, 0xAA(0b10101010)로 홀수번째 LED만 켜지도록 만듭니다. 그리고 _delay_ms(1000); 함수는 LED가 켜진 상태에서 1초 동안 지연시키는 동작을 하게 만듭니다.

```
        return 0;
}
```

프로그램이 함수에 도달하면 0을 반환하고, main 함수를 종료합니다.

두 번째로 LED가 오름차순, 내림차순을 반복하며 점등하는 동작하는 코드를 작성해야 합니다.
(Code_Up and Down)

```
#define F_CPU 16000000L
#include <avr/io.h>
#include <util/delay.h>
```

첫번째와 같이 CPU의 주파수와 AVR 라이브러리 속 헤더 파일을 사용할 수 있도록 초기값을 설정합니다.

```
int main(void)
{
        char current_pattern_value;
        char next_pattern_value = 0b11111110;
        char reverse_next_pattern_value = 0b01111111;
        int a = 1;
```

main함수를 통해 진입점을 정의하고, char 함수를 통해서 current_pattern_value라는 값을 저장할 변수를 쓴다고 하고, 오름차순으로 사용할 next_pattern_value이란 변수에 0b1111110(0xFE)이라는 값을 저장합니다.
그리고 오름차순의 역순인 내림차순으로 쓸 reverse_next_pattern_value 변수에는 0b01111111(0x7F)이라는 값을 저장합니다.
또한 int a = 1을 통해 패턴 방향을 제어하는 변수를 선언하고, 초기값은 오름차순 패턴을 나타냅니다.

```
        DDRB = 0xFF;

        while (1)
```

포트 B 핀을 0b11111111로 만들어 출력으로 설정하고, while문을 통해 프로그램이 다음 코드를 무한 반복하게끔 만듭니다.

```
        {
                current_pattern_value = next_pattern_value;
                PORTB = current_pattern_value;
```

current_pattern_value를 통해 현재 패턴에 다음 패턴의 값으로 넣어줍니다. 그리고 포트 B에 있는 현재 패턴의 값을 출력하여 LED를 점등하게 합니다.

```
                if(a==1){
                next_pattern_value = next_pattern_value << 1;
                next_pattern_value = next_pattern_value + 1;
                        if(next_pattern_value == 0b11111111){
                        a=-1;}
```

```
                    }
```

이제 if문을 통해서 a==1일 때 오름차순 패턴으로 LED가 점등될 수 있도록, << 쉬프트 함수를 사용해서 next_pattern_value를 왼쪽으로 한 비트씩 이동시킵니다. 한 비트를 이동시키면 최하위 비트는 0이 딸려나와 0b11111100이 되기 때문에 LED 두 개가 점등되는 오류가 생길 수 있습니다. 그래서 최하위비트(LSB)에 1을 더해주어 오류를 사전에 방지합니다.
이 동작이 반복되어 next_pattern_value가 최대값인 0b11111111을 만족하게되면 변수 a의 값을 -1로 설정하여 다음 내림차순 패턴으로 바뀔 수 있도록 합니다.

```
            if(a==-1) {
            reverse_next_pattern_value = reverse_next_pattern_value >> 1;
            reverse_next_pattern_value = reverse_next_pattern_value + 0b10000000;
```

오름차순이 끝나고 변수 a의 값이 -1로 변경되어서 또 다른 if문인 if(a==-1)가 시작됩니다. 이것은 내림차순 패턴의 if문으로, reverse_next_pattern_value를 오른쪽으로 한 비트씩 쉬프트시킵니다. 그러면 0b00111111으로 변하게 되는데 최상위 비트에 0이 딸려나와 아까와 같은 오류를 범할 수 있습니다. 그렇기 때문에 0b10000000을 더하여 LED가 점등되지 않도록 방지합니다.

```
            next_pattern_value = reverse_next_pattern_value;
                    if(next_pattern_value == 0b11111111){
                            a=1;
```

오름차순에서 내림차순으로 변수를 바꿔주기 위하여 next_pattern_value에 reverse_next_pattern_value라는 값을 대입해주고 작동되게끔 만듭니다. 그리고 최대값이 된다면 a=1을 출력하여 오름차순 if문 패턴으로 돌아가게 합니다.

```
                            next_pattern_value = 0b11111110;
                            reverse_next_pattern_value = 0b01111111;
                            }
```

그리고 next_pattern_value와 reverse_next_pattern_value가 최대값으로 변했기 때문에 정상적으로 처음 패턴을 반복하기 위해 초기값을 다시 넣어주어 무한 루프가 정상적으로 작동하게 만들어줍니다.

```
                    }

                    _delay_ms(100);
            }

            return 0;
    }
```

그리고 딜레이 함수를 통해 100ms 동안 지연하게 하고, return을 통해 main함수를 종료시키고 0을 반환하게 만듭니다.

이렇게 AVR Studio에서 작성한 Code_ODD or Even과 Code_Up and Down을 hex 파일로 저장하고 ISP 케이블을 ATmega128 마이크로컨트롤러에 연결하여 hex 파일을 실행시켜서 LED가 코드가 짜여진 대로 작동하는지 확인합니다.

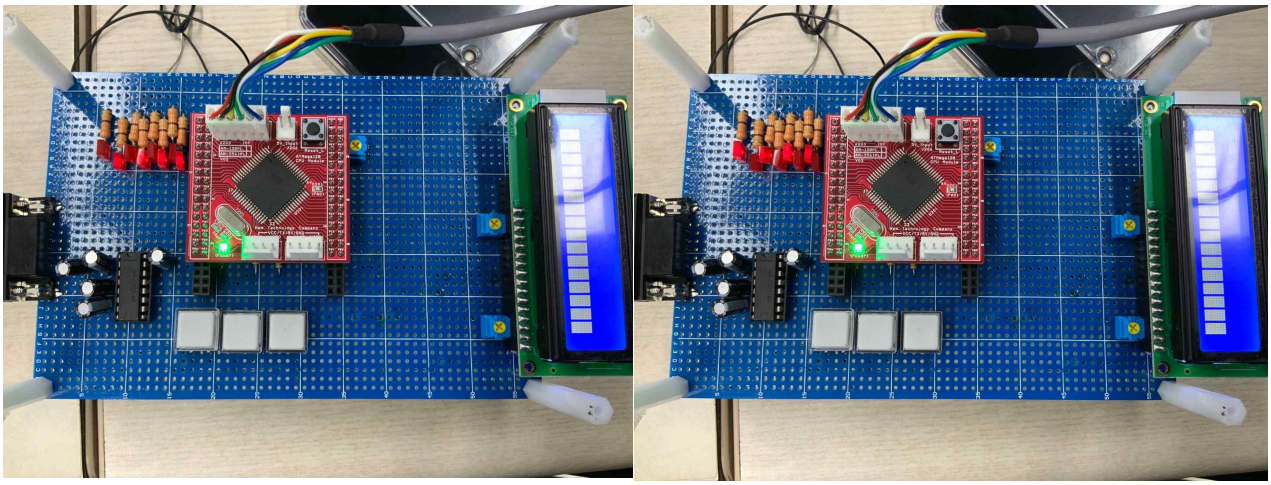
5. 측정값

완성된 ATmega128 마이크로 컨트롤러와 컴퓨터를 이용한 AVR Studio를 이용한 코드를 통해 LED를 홀수·짝수, 내림차순·오름차순처럼 각각 동작을 다르게 하여 아래와 같이 측정된 값을 얻었다.

Code_ODD or Even	LED가 짝수번째, 홀수번째로 점등하는 동작을 반복한다.
Code_Up and Down	LED가 오름차순, 내림차순으로 점등하는 동작을 반복한다.

표 3) AVR Studio를 통해 작성한 코딩 테스트 측정값

6. 실험결과



코딩(Coding)을 하고 LED가 원하는대로 동작하는 정상적인 코드를 만들었는지 알아보기 위하여 테스트를 진행하였다.

첫 번째 테스트, Code_ODD or Even(홀수·짝수)는 실제로도 LED가 홀수번째끼리, 짝수번째끼리 점등하는 것을 바로 확인하였고, 테스트에 성공했다. 하지만 Code_Up and Down(오름차순·내림차순)에서 하나를 제외한 모든 불이 꺼져야 했는데 하나를 제외한 모든 불이 켜지는 오류가 발생했다. DDRB의 값을 입력으로 설정하여 벌어진 일이었다. 이후 출력으로 바꾸니 정상적으로 하나만 불이 들어오게 되었다.

두 번째 테스트, LED가 하나만 불이 점등 되었지만, 비트가 왼쪽으로 1비트씩 쉬프트되면서 최하위 비트에서 0이 딸려와 LED가 하나씩만 점등되는게 아닌, 하나씩 더해지며 1개, 2개, 3개, 4개 … 이렇게 출력되게 되는 오류가 발생했다. 그래서 1비트씩 쉬프트시킬 때마다 최하위 비트에 1을 더해주어 오류를 수정했다. 이제 오름차순은 완벽하게 작동하지만, 내림차순으로 가는 방법을 찾지못했다.

세 번째 테스트, 내림차순을 진행하기 위해 변수 a를 만들고 오름차순 패턴이 종료되면 a에 -1 값을 넣고, 내림차순 패턴이 a == -1일 때 시작되도록 if문을 설정했다. 그리고 오름차순인 next_pattern_value와 다른 변수인

reverse_next_pattern_value를 만들었다. 그리고 이번엔 반대인 오른쪽으로 한 비트씩 이동시키며, 최상위 비트에 1을(0b10000000) 더해주었더니, 내림차순도 작동하게 되었다. 하지만 while문을 통해 동작하지 못했다.

마지막 테스트, while문을 통해 무한 반복이 되지 않았던 이유는 next_pattern_value와 reverse_next_pattern_value가 초기값이 아닌 최대값으로 변해있어 if문이 작동하지 않았다. 그래서 오름차순 내림차순의 한 주기가 끝나면 next_pattern_value와 reverse_next_pattern_value의 값을 초기값으로 변경해주어, if문에서 정상 작동하게 만들었다. 그래서 최종적으로 Code_Up and Down(오름차순·내림차순)이 완전히 정상 작동하는 모습을 관찰할 수 있었다.

7. 결과에 대한 논의

이번 실습에서는 조교님께 질문하는 대신 같은 공동실험자인 조원과 함께 탐구하고 의논해보는 시간을 깊게 가졌다. 확실히 코딩 언어를 만져본 조원님은 수월하게 진행하시는 편이었다. 반면에 나는 아쉽게도 코딩에 지식이 전무했기 때문에 어떻게 코드가 작동하는지 찾아보고, 친절하게도 조원분이 알려주신 지식을 통해서 진행했다. 코드를 많이 사용할 것 같은데 기본적인 비트연산자를 모르고 실습에 참여했다는 사실이 많이 미흡했던거 같다. 교수님께서 E-루리에 제공해주신 교재인 마프_Part04_개발환경+C언어로 코드의 작동 원리 및 함수들을 공부하고 와서 실습을 원활히 따라갈 수 있도록 할 예정이다.

그래도 실습이 잘됐던 부분은 조원분과 함께 오래 고민해도 풀지 못했던 파트의 코드를 성공적으로 작성했다는 점이다. 내림차순을 이동시키고 점등하는 부분이었는데, 오름차순은 최하위 비트에 1을 더하면 되어서 비교적 쉽게 작동했지만, 내림차순은 어떻게 더해야 할지 구상이 되지 않았었다. 내림차순은 초깃값과 이동 방향이 오름차순의 역순이니깐 최상위 비트에 1을 더하면 어떨까? 라는 구상을 통해 코드를 작성 후 테스트했더니 정상 작동하는 모습을 보고 나름 뿌듯했다.

우리가 직접 작성한 코드를 통해 직접 제작한 마이크로 컨트롤러 보드의 LED를 원하는대로 작동시킬 수 있다는 점이 이 실험의 의의다.
앞으로 LED를 제외한 또 어떤 동작을 하게 만들 수 있을지 더욱 탐구심을 일으켰다.

8. 결론

ATmega128 마이크로 컨트롤러 보드가 AVR Studio에서 비트 연산자, 변수, DDR, PORT 같은 함수를 사용하여 작성한 코드에 의해, LED가 홀수·짝수 혹은 오름차순·내림차순과 같은 동작을 코드에서 의도한 것과 같이 동작시킬 수 있었다.

9. 참고문헌 및 출처

(교재) 김도인 교수님, 「실습교재5」, 강원대학교, 전기전자공학과, (13~14p)
(교재) 김도인 교수님 [2024] 마프_Part 04_개발환경+C언어_ver2, “, ”, (14~39p)